Brain-Computer Interfaces (25-26) - Assignment #2

Eye State Prediction from EEG Signals Using Classical Machine Learning

orvile | Python / Google Colab | Public repository under the MIT License

Abstract

This report presents a classical machine-learning pipeline for binary eye-state prediction from EEG signals. The task uses the EEG Eye State dataset introduced by Rösler and Suendermann, where 14 Emotiv EPOC EEG channels are used to classify each sample as eye open (0) or eye closed (1). The final selected dataset representation was raw_sample with 14,974 samples after robust outlier removal. A preliminary model screening was performed with LazyPredict and manual cross-validation, followed by Optuna hyperparameter optimization. The best final model was Label Propagation with an RBF kernel, achieving 97.3957% hold-out accuracy, 97.3611% balanced accuracy, and 97.3676% macro F1-score. The result is comparable to the original KStar baseline reported by Rösler and Suendermann and to later ensemble-based studies.

1. Dataset and Problem Definition

The EEG Eye State dataset is a multivariate, sequential, time-series classification dataset. It contains EEG values from 14 channels and a target label indicating the eye state. The original recording lasted approximately 117 seconds and was collected from one subject using an Emotiv EEG Neuroheadset. The eye state was manually annotated from synchronized video frames. In this assignment, 0 represents eye open and 1 represents eye closed. The main objective is to build a reliable classical machine-learning classifier and compare its performance with previously reported results in the literature [1], [2].

The final experiment used the raw-sample representation rather than the window-based representation. This decision was made because the original dataset is distributed as row-wise 14-channel EEG samples, and most reported literature values are directly comparable under this representation. The window-based experiment was kept as an exploratory signal-processing variant, but its group-aware cross-validation performance was unstable and much lower, so it was not selected as the final model setting.

2. Literature-Based Model Motivation

The literature strongly suggested that instance-based and ensemble-style methods should be included. Rösler and Suendermann tested 42 classifiers and reported KStar as the best method with a 2.7% error rate, corresponding to approximately 97.3% accuracy [1]. Sahu et al. found that instance-based methods such as IB1 and IBK performed well on the same dataset [3]. Al-Taei later proposed a voting ensemble combining KStar and Random Forest and reported 97.27% accuracy [4]. Nilashi et al. used Learning Vector Quantization with Bagged Trees and reported 94.31% accuracy [5]. Based on these findings, the experiments included KNN-like instance-based models, Label Propagation / Label Spreading, Extra Trees, Random Forest, XGBoost, LightGBM, Bagging, SVC, and simpler baselines.

Study	Method	Accuracy	Note
Rösler & Suendermann, 2013	KStar / Weka	0.9730	Original study; 10-fold CV
Al-Taei, 2017	Vote(KStar + RF)	0.9727	Voting ensemble
Sahu et al., 2015	IB1	0.9451	Instance-based learning
Nilashi et al., 2023	LVQ + Bagged Trees	0.9431	Hybrid clustering + ensemble trees
Sahu et al., 2015	IBK-3	0.9342	KNN-like method
Sahu et al., 2015	Random Forest	0.8927	Tree ensemble

3. Methodology and Code Explanation

The notebook was designed in Google Colab because the project uses several Python packages that are easier to install and reproduce in a managed runtime, including scipy for ARFF loading, scikit-learn for classical ML models, LazyPredict for fast model screening, Optuna for hyperparameter optimization, and optional SHAP/LIME explainability modules. Colab also avoids local environment conflicts and allows the instructor to reproduce the workflow from the public GitHub repository.

Step	Explanation
Data loading	Download ARFF.gz, parse with scipy.io.arff, convert to DataFrame, and save a CSV copy for inspection.
Type cleaning	Convert eyeDetection from ARFF byte/string values to integer labels: 0=open, 1=closed.
Outlier handling	Use robust MAD-based row filtering; final raw_sample shape is (14974, 14).
Model screening	Use LazyPredict and manual CV to select promising model families before tuning.
Scaling	Use StandardScaler for scale-sensitive models; add RobustScaler variants for KNN/SVC; tree models are left unscaled.
Window experiment	Create statistical features only inside homogeneous 0/1 segments to avoid mixed-label windows.
Final tuning	Tune selected models with Optuna; optimize accuracy and report balanced accuracy, macro F1, precision, recall, and confusion matrix.

4. Experimental Results

The first raw-sample cross-validation screening showed that graph-based methods and KNN-like models were the strongest candidates. Label Spreading RBF achieved 0.971484 mean accuracy, Label Propagation RBF achieved 0.971417, and KNN Distance + RobustScaler achieved 0.964138. This matched the literature trend where instance-based and neighborhood-sensitive methods are strong for this dataset.

Model	CV Accuracy	Bal. Acc.	Macro F1
Label Spreading RBF	0.971484	0.971024	0.971175
Label Propagation RBF	0.971417	0.970977	0.971109
KNN Distance + RobustScaler	0.964138	0.963339	0.963728
KNN Distance	0.961332	0.960683	0.960908
Extra Trees	0.952317	0.949966	0.951620
SVC RBF	0.935488	0.933870	0.934680

LazyPredict produced a similar ranking in a fast hold-out screening: LabelPropagation reached 0.968948 accuracy, LabelSpreading reached 0.968614, and KNeighborsClassifier reached 0.962938. This justified selecting graph-based label methods and KNN for the Optuna stage. The window-based model screening, however, showed poor and highly variable group-aware results, with Label Propagation RBF and Label Spreading RBF around 0.535899 mean accuracy and high standard deviation. Therefore, the final report focuses on the raw-sample experiment.

Tuned model	Best CV Accuracy	Best parameters summary
Label Propagation RBF	0.974823	gamma≈9.805, max_iter≈962
Label Spreading RBF	0.970883	gamma≈28.376, alpha≈0.252
KNN Distance + RobustScaler	0.971951	n_neighbors=1, p=1

Metric	Value
Dataset	raw_sample
Model	Label Propagation RBF
Hold-out accuracy	0.973957
Balanced accuracy	0.973611
Macro F1-score	0.973676
Macro precision	0.973743
Macro recall	0.973611

The final hold-out classification report also showed balanced behavior across both classes. Eye Open (0) achieved approximately 0.98 precision, recall and F1-score over 1651 test samples. Eye Closed (1) achieved approximately 0.97 precision, recall and F1-score over 1344 test samples. This indicates that the final model did not obtain high accuracy simply by favoring the majority class.

Study	Method	Accuracy	Note
Our experiment	Label Propagation RBF	0.973957	Hold-out after Optuna
Rösler & Suendermann, 2013	KStar / Weka	0.973000	Original 10-fold CV
Al-Taei, 2017	Vote(KStar + RF)	0.972700	Voting ensemble
Sahu et al., 2015	IB1	0.945100	Instance-based
Nilashi et al., 2023	LVQ + Bagged Trees	0.943100	Hybrid ensemble

5. Discussion and Conclusion

The final result of 97.3957% accuracy is slightly higher than the original KStar result reported by Rösler and Suendermann and close to the KStar + Random Forest voting ensemble reported by Al-Taei. The improvement should not be overinterpreted because the exact evaluation protocol differs across studies; the original study used 10-fold cross-validation, whereas the final number here is a hold-out result after model selection and Optuna tuning. Still, the result supports the same general conclusion: the EEG Eye State dataset is highly suitable for neighborhood-sensitive and graph-based classical machine-learning methods.

The main limitation is that the dataset comes from a single subject and one continuous recording session. Therefore, the model demonstrates within-dataset classification performance rather than subject-independent generalization. In a more realistic BCI setting, additional subjects, session-level validation, and subject-independent splits would be necessary. Nevertheless, for the assignment objective, the pipeline is reproducible, justified by prior literature, and restricted to classical ML rather than ANN/deep learning.

Repository, License and Reproducibility

The GitHub repository is intended to be public and released under the MIT License. The MIT License applies to the authored code and project documentation, allowing others to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the software as long as the copyright and license notice are preserved. The dataset and cited papers remain under their own original licenses and citation requirements. A public repository is appropriate here because the work is educational, reproducible, and based on publicly available data.

Suggested GitHub title: EEG Eye State Prediction with Classical Machine Learning. Suggested GitHub description: Classical ML pipeline for EEG eye-state prediction using LazyPredict screening, Optuna tuning, and literature comparison on the EEG Eye State dataset.

Project Repository

The source code and project files are publicly available on GitHub:

https://github.com/BoranT-3000/EEG-Eye-State-Prediction-with-Classical-Machine-Learning

The repository is shared under the MIT License to make the implementation transparent, reusable, and accessible for educational purposes.

References

[1] O. Rösler and D. Suendermann, "A First Step towards Eye State Prediction Using EEG," AIHLS, 2013. https://suendermann.com/su/pdf/aihls2013.pdf
[2] UCI Machine Learning Repository, "EEG Eye State" dataset. https://archive.ics.uci.edu/ml/datasets/EEG+Eye+State
[3] M. Sahu et al., "Performance Evaluation of Different Classifier for Eye State Prediction Using EEG Signal," International Journal of Knowledge Engineering, 2015. https://www.ijke.org/vol1/24-E002.pdf
[4] A. Al-Taei, "Ensemble Classifier for Eye State Classification using EEG Signals," 2017. https://arxiv.org/abs/1709.08590
[5] M. Nilashi et al., "Electroencephalography (EEG) eye state classification using learning vector quantization and bagged trees," Heliyon, 2023. https://doi.org/10.1016/j.heliyon.2023.e15258